

**Informe del Proyecto Integrador: Decisiones Éticas y Arquitectura Segura  
en Sistemas de Contenido Multimedia**

Equipo de Desarrollo

Facultad de Ingeniería

Universidad Internacional del Ecuador (UIDE)

Ética y Responsabilidad Profesional

Gabriel Rivadeneira

15 de junio de 2026

## Resumen

El presente informe documenta el análisis ético y la implementación técnica de cinco decisiones fundamentales en el desarrollo de un clon de la plataforma Pinterest, desarrollado como proyecto integrador de cuarto nivel. Cada decisión se examina a la luz de marcos teóricos de la ética normativa –deontología kantiana, utilitarismo de Mill, y los principios de la *Association for Computing Machinery* (ACM)– con énfasis en el Código de Ética y Conducta Profesional ACM (2018). Se presentan: (a) una matriz de decisiones ético-técnicas que vincula cada funcionalidad del sistema con su riesgo, criterio y solución implementada; (b) un análisis profundo de cinco dilemas críticos, incluyendo la restricción de edad, validación de contraseñas, filtrado MIME, limitación de tasa y control de propiedad intelectual; (c) una tabla de implementación verificable con referencias directas al código fuente; y (d) las normas del sistema derivadas de estas decisiones. Los resultados demuestran que es posible codificar principios éticos en infraestructura software mediante técnicas como validación asimétrica de *magic bytes*, *rate limiting*, hashing con *bcrypt*, y autenticación por cookies *HttpOnly*, sin sacrificar la funcionalidad central de la plataforma. Se concluye que la ingeniería de software no es axiológicamente neutral: cada decisión arquitectónica constituye una postura moral traducible a lógica computacional.

**Informe del Proyecto Integrador: Decisiones Éticas y Arquitectura Segura  
en Sistemas de Contenido Multimedia**

**Índice**

|                                                                             |          |
|-----------------------------------------------------------------------------|----------|
| <b>Índice</b>                                                               | <b>3</b> |
| <b>Introducción</b>                                                         | <b>5</b> |
| Contexto del proyecto . . . . .                                             | 5        |
| Planteamiento del problema ético . . . . .                                  | 5        |
| Declaración de uso de inteligencia artificial . . . . .                     | 5        |
| <b>Marco Teórico</b>                                                        | <b>6</b> |
| Deontología kantiana . . . . .                                              | 6        |
| Utilitarismo de Mill . . . . .                                              | 6        |
| Código de Ética ACM . . . . .                                               | 6        |
| Principio de beneficencia . . . . .                                         | 7        |
| Síntesis del marco . . . . .                                                | 7        |
| <b>Metodología</b>                                                          | <b>7</b> |
| Diseño del sistema . . . . .                                                | 7        |
| Infraestructura tecnológica . . . . .                                       | 8        |
| Procedimiento de evaluación ética . . . . .                                 | 8        |
| <b>Resultados</b>                                                           | <b>8</b> |
| Matriz de decisiones ético-técnicas . . . . .                               | 8        |
| Análisis de decisiones . . . . .                                            | 8        |
| Decisión 1: Restricción de edad vs. expansión comercial . . . . .           | 8        |
| Decisión 2: Seguridad de credenciales vs. usabilidad . . . . .              | 9        |
| Decisión 3: Libertad de publicación vs. integridad del ecosistema . . . . . | 10       |
| Decisión 4: Acceso ilimitado vs. equidad en recursos . . . . .              | 10       |
| Decisión 5: Control de propiedad intelectual vs. censura . . . . .          | 11       |
| Matriz de implementación verificable . . . . .                              | 11       |

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>Discusión</b>                                       | <b>11</b> |
| Interpretación de los resultados . . . . .             | 11        |
| Implicaciones para la ingeniería de software . . . . . | 12        |
| Limitaciones . . . . .                                 | 13        |
| Trabajo futuro . . . . .                               | 13        |
| Normas derivadas del sistema . . . . .                 | 13        |
| Reglas de uso . . . . .                                | 14        |
| Políticas de contenido . . . . .                       | 14        |
| Responsabilidades del usuario . . . . .                | 14        |
| <b>Conclusiones</b>                                    | <b>15</b> |
| <b>Referencias</b>                                     | <b>16</b> |

## Introducción

### Contexto del proyecto

El desarrollo de plataformas web de contenido generado por el usuario (*User Generated Content*, UGC) plantea dilemas éticos que trascienden la mera corrección técnica. Cuando un sistema permite a cualquier persona registrarse, publicar imágenes e interactuar con otros usuarios, cada decisión arquitectónica —desde la validación de una contraseña hasta el formato de una cookie— se convierte en una postura moral con consecuencias reales sobre la privacidad, la seguridad y el bienestar de los usuarios (Floridi & Cowls, 2019).

Este informe analiza el caso del **Pinterest Clone**, una aplicación web de contenido multimedia desarrollada como Proyecto Integrador de 4to Nivel de la Facultad de Ingeniería de la UIDE. La plataforma emplea un backend en FastAPI (Python) con autenticación JWT, almacenamiento en AWS S3, y un frontend en React con Vite. A lo largo de su ciclo de desarrollo, el equipo enfrentó cinco conflictos éticos fundamentales que requirieron decisiones explícitas y justificables.

### Planteamiento del problema ético

¿Cómo puede un equipo de desarrollo garantizar que las decisiones arquitectónicas de una plataforma de contenido multimedia se alineen con principios éticos reconocidos, sin sacrificar la funcionalidad ni la experiencia de usuario? Esta pregunta guía el presente informe. De ella se desprenden cinco sub-preguntas correspondientes a cada dilema identificado, que se detallan en la sección de Resultados.

### Declaración de uso de inteligencia artificial

De acuerdo con las políticas de integridad académica y transparencia tecnológica, se declara formalmente que para el desarrollo de este proyecto integrador se utilizó un **25 % de Inteligencia Artificial**. La IA fue empleada bajo el principio de *augmentación humana* (*human augmentation*), exclusivamente para: la depuración de errores sintácticos y de concurrencia en FastAPI (Python); la optimización asíncrona de los algoritmos de lectura de *Magic Bytes* en el backend; y la generación estructurada de componentes reutilizables en React. El diseño arquitectónico de alto nivel, el

discernimiento ético, la configuración de la infraestructura en la nube (AWS S3), y la definición de políticas de seguridad fueron concebidos, programados y evaluados de manera autónoma e íntegra por los autores, conforme al principio ACM 1.3 (*Ser honesto y digno de confianza*) y a los estándares de integridad académica de la institución.

### Marco Teórico

El análisis de las decisiones del proyecto se estructura en torno a cuatro pilares normativos de la ética filosófica y profesional, que se detallan a continuación. La Figura 1 (en Resultados) presenta cuantitativamente la relevancia de cada principio del Código ACM para las cinco decisiones implementadas.

#### Deontología kantiana

La ética del deber, formulada por Kant (1785/2012), sostiene que ciertas acciones son intrínsecamente correctas o incorrectas independientemente de sus consecuencias. Este marco normativo establece que los seres humanos deben ser tratados como fines en sí mismos, nunca como medios. En el contexto del desarrollo de software, la deontología kantiana impone un deber incondicional de proteger a los usuarios vulnerables, particularmente menores de edad, independientemente del costo para el crecimiento de la plataforma. Este principio fue el eje de la Decisión 1 (restricción de edad).

#### Utilitarismo de Mill

El utilitarismo clásico de Mill (1861/2001) propone que la corrección moral de una acción se determina por su contribución a la maximización del bienestar agregado. Sin embargo, Mill advierte que la calidad de los placeres importa tanto como la cantidad. Este marco ayudó a evaluar las compensaciones entre la libertad del usuario y la seguridad colectiva, particularmente en las Decisiones 2 (usabilidad vs. seguridad) y 4 (acceso ilimitado vs. equidad en recursos).

#### Código de Ética ACM

El Código de Ética y Conducta Profesional de la Association for Computing Machinery (ACM, 2018) proporciona un marco operativo para ingenieros de software. Los principios más relevantes para este proyecto fueron: 1.1 (*Contribuir a la sociedad y*

*al bienestar humano*), 1.2 (*Evitar el daño a otros*), 1.3 (*Ser honesto y digno de confianza*), 1.4 (*Ser justo y tomar acción para no discriminar*), y 1.5 (*Respetar la privacidad*). Mittelstadt et al. (2016) argumentan que estos principios permiten traducir valores abstractos a criterios de diseño verificables.

### **Principio de beneficencia**

Beauchamp y Childress (2019) definen la beneficencia como la obligación de producir beneficios netos sobre daños. Este principio, tomado de la bioética pero aplicable al diseño de sistemas, fue particularmente relevante para evaluar las compensaciones entre seguridad y usabilidad en las decisiones 2 (validación de contraseñas) y 4 (rate limiting), donde la restricción impuesta genera un beneficio colectivo que supera el costo individual de la fricción.

### **Síntesis del marco**

En conjunto, estos cuatro marcos proporcionan un sistema de evaluación multi-criterio para cada decisión técnica. La deontología establece los límites mínimos (*lo que nunca debe hacerse*), el utilitarismo evalúa las compensaciones (*lo que maximiza el bienestar*), el código ACM operacionaliza los principios en criterios de ingeniería, y la beneficencia exige un balance neto positivo. Ningún marco opera de forma aislada; cada decisión fue evaluada contra los cuatro antes de su implementación, como se documenta en las subsecciones 4.2.1 a 4.2.5 y se visualiza en la Figura 1.

## **Metodología**

### **Diseño del sistema**

El Pinterest Clone fue desarrollado siguiendo una arquitectura de tres capas: frontend en React con Vite, backend en FastAPI (Python 3.10+), y almacenamiento en AWS S3. La comunicación entre capas utiliza una API REST asíncrona con autenticación JWT transmitida mediante cookies HttpOnly para mitigar riesgos de XSS. Esta arquitectura se ilustra parcialmente en el diagrama de la Figura 4 para el flujo de subida de archivos.

## Infraestructura tecnológica

El backend implementa hashing asimétrico con `bcrypt` para credenciales (`security.py`), validación binaria de archivos mediante detección de *magic bytes* (`pins.py`), limitación de tasa con middleware `slowapi` (`limiter.py`), y registro de auditoría estructurado en JSON (`audit_log.py`). El frontend incorpora validación de edad en el registro (`RegisterPage.jsx`) y un indicador visual de fortaleza de contraseña en tiempo real. Los diagramas de flujo en las Figuras 2, 3 y 4 detallan visualmente estos procesos.

## Procedimiento de evaluación ética

Cada decisión técnica fue evaluada siguiendo un procedimiento de cuatro pasos: (1) identificación del conflicto ético subyacente, (2) análisis de las teorías normativas en tensión, (3) justificación de la decisión mediante criterios ACM, y (4) implementación técnica verificable con referencia al código fuente. Este procedimiento se aplicó a los cinco dilemas documentados en la sección de Resultados.

## Resultados

### Matriz de decisiones ético-técnicas

Para garantizar un entorno digital seguro y alineado con el Código de Ética de la ACM, se implementaron cinco decisiones fundamentales en el flujo del sistema. La Tabla 1 resume la matriz completa de control ético-técnico, vinculando cada funcionalidad con su riesgo, criterio y solución en código. La Figura 1 muestra el mapa de calor de los principios ACM aplicados.

### Análisis de decisiones

#### *Decisión 1: Restricción de edad vs. expansión comercial*

**Conflicto.** El deseo corporativo de maximizar la base de usuarios y el crecimiento de la plataforma (*Monthly Active Users*) entra en conflicto directo con la responsabilidad moral de proteger a menores de edad frente a contenidos multimedia generados por terceros que podrían no estar filtrados.

**Teorías en tensión.** El utilitarismo de mercado sugiere minimizar la fricción en



el *onboarding* para maximizar el bienestar agregado (más usuarios = más valor). Sin embargo, la deontología kantiana impone un deber incondicional de proteger a los menores, independientemente del costo para el crecimiento. La ética de la virtud añade que una plataforma justa no debería beneficiarse de la vulnerabilidad de sus usuarios más frágiles.

**Decisión.** Se priorizó la ética del deber. La plataforma no cuenta actualmente con un equipo masivo de moderadores de contenido en tiempo real; por lo tanto, la única forma técnica y ética de mitigar el riesgo es cortar el acceso en la etapa de registro.

**Implementación.** La Figura 2 presenta el diagrama de flujo de la decisión. En el frontend, `RegisterPage.jsx:20-24` calcula la fecha máxima usando `getFullYear() - 18` para restringir el calendario HTML. En el backend, `models.py:15` almacena `fecha_nacimiento` como campo `date` en la tabla `users`.

### ***Decisión 2: Seguridad de credenciales vs. usabilidad***

**Conflicto.** Exigir contraseñas robustas incrementa la seguridad pero introduce fricción: los usuarios pueden abandonar el registro si el proceso es percibido como complejo o tedioso.

**Teorías en tensión.** El principio de responsabilidad profesional (ACM 1.3) obliga al desarrollador a custodiar los datos del usuario con el más alto estándar disponible. La usabilidad (*beneficencia*) sugiere no imponer cargas excesivas.

**Decisión.** Se implementó un indicador visual de fortaleza de contraseña en tiempo real que proporciona *feedback* inmediato al usuario. Esto transforma la restricción de seguridad en una herramienta educativa, alineándose con el principio ACM 1.1 (*contribuir al bienestar humano*).

**Implementación.** La Figura 3 presenta el ciclo completo. En el frontend, `RegisterPage.jsx:5-11` implementa un patrón RegEx con 5 criterios (longitud, mayúscula, minúscula, número, símbolo) con barra de progreso visual. En el backend, `security.py:13-16` ejecuta `hash_password()` usando `bcrypt.gensalt()` con trunco a 72 bytes y `hashpw` para hashing asíncrono.

***Decisión 3: Libertad de publicación vs. integridad del ecosistema***

**Conflicto.** Facilitar la libertad de expresión y la facilidad de uso *versus* garantizar la seguridad del ecosistema frente a archivos maliciosos (malware/ransomware oculto en imágenes mediante esteganografía).

**Teorías en tensión.** La autonomía del usuario respalda el derecho a subir cualquier formato. Sin embargo, la no maleficencia (ACM 1.2) establece la obligación del desarrollador de prevenir daños activos.

**Decisión.** Se adoptó validación a nivel de *magic bytes*. El sistema no confía en la extensión provista por el cliente: FastAPI intercepta el flujo, lee los primeros bytes en memoria y comprueba la firma criptográfica del archivo.

**Implementación.** La Figura 4 presenta la arquitectura. `pins.py:20-29` implementa `_detect_mime()` que examina firmas hexadecimales (JPEG: FF D8 FF, PNG: 89 50 4E 47, GIF: 47 49 46 38, WebP: 52 49 46 46 + 57 45 42 50). `pins.py:66-92` integra la verificación de extensión, detección MIME y límite de tamaño (10 MB). `storage_service.py:47-49` sanitiza el nombre con UUIDv4.

***Decisión 4: Acceso ilimitado vs. equidad en recursos***

**Conflicto.** Un usuario malintencionado puede saturar la API con peticiones automatizadas (*web scraping, denial of service*), degradando la experiencia de todos los demás usuarios legítimos.

**Teorías en tensión.** La libertad de acceso sugiere que no deberían existir restricciones. Sin embargo, la justicia distributiva (Rawls, 1971) exige que los recursos compartidos se asignen de manera equitativa. El principio ACM 1.2 (*evitar el daño*) respalda la limitación como medida de prevención de daño colectivo.

**Decisión.** Se implementó *rate limiting* con el middleware `slowapi`, restringiendo los endpoints críticos a 10 peticiones por minuto por dirección IP.

**Implementación.** `limiter.py:1-4` instancia el `Limiter` con clave por dirección IP remota. `main.py:72-78` vincula el limiter al estado de la aplicación. `pins.py:155` aplica el decorador `@limiter.limit("10/minute")` en el endpoint `create_pin`.

***Decisión 5: Control de propiedad intelectual vs. censura***

**Conflicto.** Permitir que cualquier usuario elimine cualquier publicación facilita la moderación pero viola el derecho del creador original. Al mismo tiempo, permitir solo al administrador eliminar contenidos crea un cuello de botella.

**Teorías en tensión.** La autonomía del creador sobre su obra se enfrenta al deber de justicia de la comunidad de eliminar contenido dañino.

**Decisión.** Se adoptó un modelo híbrido: el autor original o un administrador del sistema pueden eliminar un pin o comentario.

**Implementación.** `services.py:90-97` compara `pin.autor_id` con `user.id` y verifica `user.es_admin`. `services.py:123-129` aplica la misma lógica para comentarios. `audit_log.py:36-37` registra intentos no autorizados.

**Matriz de implementación verificable**

La Tabla 2 expone las medidas técnicas concretas desarrolladas para sostener los valores promulgados, con referencias verificables al código fuente. La Figura 5 compara la severidad del riesgo, la efectividad de la mitigación y el riesgo residual.

**Discusión****Interpretación de los resultados**

Como se reportó en la sección de Resultados, las cinco decisiones implementadas logran reducir el riesgo ético a niveles residuales aceptables. La Figura 5 muestra que las puntuaciones residuales oscilan entre 1.0 y 3.5 en una escala de 0 a 5, lo que indica que ninguna mitigación es absoluta pero todas reducen significativamente el riesgo original. La mitigación más efectiva corresponde a la validación de *magic bytes* (Decisión 3, diagrama en Figura 4), que alcanza una efectividad de 5.0 al eliminar por completo la posibilidad de subir archivos maliciosos camuflados como imágenes. La mitigación con menor efectividad absoluta es el *rate limiting* (Decisión 4), con 4.2, debido a que un atacante distribuido (DDoS) podría eludir la restricción por IP.

El mapa de calor de la Figura 1 revela que el principio ACM 1.2 (*no maleficencia*) es el más transversal, apareciendo con alta relevancia en las cinco decisiones. Esto es consistente con la naturaleza preventiva del sistema: la mayoría de las

mitigaciones actúan antes de que ocurra un daño potencial. Por el contrario, principios como 1.6 (*reputación*) y 2.1 (*competencia profesional*) presentan baja relevancia, lo que refleja el alcance del proyecto como desarrollo académico y no como producto comercial.

Estos hallazgos son consistentes con el marco de Nissenbaum (2009) sobre privacidad contextual: la efectividad de una mitigación depende no solo de su corrección técnica sino de su adecuación al contexto social de uso. Por ejemplo, la restricción de edad (Decisión 1, Figura 2) es técnicamente simple –una validación de fecha– pero éticamente robusta porque respeta el principio de autonomía del menor al prevenir su exposición antes de que pueda consentir.

### Implicaciones para la ingeniería de software

Los resultados de este informe, resumidos en la Tabla 1 y la Tabla 2, respaldan la tesis de Maner (1996) de que la tecnología de la información plantea problemas éticos *únicos* que no se reducen a la ética aplicada general. La necesidad de decidir en tiempo de compilación –antes de que ocurra el daño– distingue a la ética del software de otros dominios. Cada decisión documentada en las subsecciones 4.2.1 a 4.2.5 fue implementada en código antes de que un solo usuario interactuara con el sistema, lo que convierte al desarrollador en un *diseñador moral preventivo*.

Shneiderman (2020) propone que los sistemas confiables requieren tanto corrección técnica como transparencia ética. La matriz de implementación verificable (Tabla 2) responde directamente a esta exigencia: cada fila vincula un principio abstracto con una línea de código específica, permitiendo la verificación independiente por terceros. Los diagramas de flujo (Figuras 2, 3 y 4) complementan esta transparencia al visualizar la lógica ética incrustada en la arquitectura.

La comparación riesgo-mitigación de la Figura 5 sugiere una relación que merece investigación futura: las mitigaciones más efectivas son aquellas que operan en la capa de *infraestructura* (validación binaria, hashing) en lugar de en la capa de *aplicación* (rate limiting, control de propiedad). Esto es consistente con el principio de *defense in depth* en seguridad informática, y su extensión a la ética del software merece un análisis sistemático adicional.

## Limitaciones

Este análisis presenta limitaciones que deben reconocerse explícitamente. Primero, las decisiones éticas documentadas en la Tabla 1 corresponden al estado actual del sistema y no contemplan escenarios de escala masiva (millones de usuarios concurrentes) donde los dilemas –particularmente el rate limiting y la moderación de contenido– podrían adquirir nuevas dimensiones. Segundo, el marco de evaluación se ha centrado en la ética normativa occidental; futuras iteraciones podrían beneficiarse de perspectivas adicionales como la ética del cuidado (Gilligan, 1982) o enfoques interculturales. Tercero, las puntuaciones de riesgo y mitigación presentadas en la Figura 5 son evaluaciones del equipo de desarrollo y no han sido validadas por un comité de ética externo ni mediante estudios de usuario, lo que constituye una línea de trabajo futuro inmediato.

## Trabajo futuro

Los resultados de este informe abren múltiples líneas de investigación y desarrollo. En el plano técnico, queda pendiente la implementación de un sistema automatizado de moderación de contenido basado en visión computacional, que permitiría extender el principio de no maleficencia (ACM 1.2) a la detección proactiva de contenido prohibido. En el plano ético-metodológico, la replicación de esta metodología de evaluación en otros proyectos –utilizando la matriz de la Tabla 1 como plantilla– permitiría construir un cuerpo de evidencia comparativa sobre la efectividad de distintas estrategias de mitigación. Finalmente, la extensión del marco a decisiones de infraestructura como el consumo energético de los servidores AWS S3, alineándose con el principio ACM 1.1 de contribución al bienestar humano a través de la sostenibilidad ambiental.

## Normas derivadas del sistema

Las decisiones ético-técnicas implementadas en las secciones previas (ver matriz en Tabla 1 y mitigaciones en Tabla 2) se traducen en normas concretas de convivencia digital que rigen la plataforma. El mapa de calor de la Figura 1 y la comparación riesgo-mitigación de la Figura 5 sustentan empíricamente estas normas.

### ***Reglas de uso***

**Registro veraz.** Los usuarios deben proporcionar información real durante su registro, incluyendo su fecha de nacimiento, confirmando ser mayores de edad (18+ años). El sistema valida este requisito tanto en el frontend como en el backend.

**Respeto a la comunidad.** Queda estrictamente prohibido el uso de la sección de comentarios para hostigamiento, discurso de odio, *spam* o cualquier forma de violencia digital.

**Uso personal e intransferible.** La cuenta es personal e intransferible. El usuario es responsable de mantener la confidencialidad de sus credenciales.

### ***Políticas de contenido***

**Propiedad intelectual.** Los usuarios solo pueden subir contenido multimedia del cual posean los derechos de autor o cuenten con permisos explícitos. Solo el autor o un administrador puede eliminar un Pin.

**Contenido prohibido.** No se tolerará la publicación de material explícito (NSFW), violencia gráfica, promoción de actividades ilegales o imágenes que vulneren la privacidad de terceros (*doxing*).

**Moderación.** La plataforma se reserva el derecho, a través de sus administradores, de eliminar cualquier Pin o comentario que viole estas políticas, en cumplimiento del deber de no maleficencia.

### ***Responsabilidades del usuario***

**Seguridad de la cuenta.** El usuario es responsable de todas las actividades que ocurran bajo su sesión. El sistema registra todos los eventos de inicio de sesión mediante un logger de auditoría JSON (`audit_log.py`).

**Reporte de vulnerabilidades.** Si el usuario detecta un comportamiento anómalo o contenido prohibido, es su responsabilidad moral reportarlo a los administradores.

**Uso ético de la API.** Queda prohibido el uso de bots, *scrapers* o ingeniería inversa. El rate limiting (10 peticiones/minuto) es la barrera técnica que respalda esta política.

## Conclusiones

El desarrollo del clon de Pinterest ratifica que la programación no es una disciplina axiológicamente neutral. Cada línea de código que restringe un acceso, filtra un archivo o limita una petición es una postura moral traducida a lógica booleana. A lo largo de este informe se han documentado cinco decisiones ético-técnicas fundamentales, cada una justificada desde marcos normativos establecidos y materializada en código verificable.

Las contribuciones de este trabajo son tres. **Primero**, se demuestra que es posible aplicar el Código de Ética de la ACM (2018) de manera operativa en un proyecto software concreto, vinculando principios abstractos (*no maleficencia*, *justicia distributiva*) a implementaciones técnicas específicas (*magic bytes*, *rate limiting*, *cookies* *HttpOnly*). **Segundo**, la matriz de decisiones ético-técnicas (Tabla 1) y la matriz de implementación verificable (Tabla 2) constituyen un modelo replicable para otros proyectos que enfrenten dilemas similares. **Tercero**, la arquitectura resultante —desplegada en FastAPI, React y AWS S3— establece un sistema que no solo es técnicamente solvente, sino que defiende activamente la privacidad, la seguridad y la integridad de sus usuarios frente a las amenazas del entorno digital contemporáneo.

Este análisis presenta limitaciones que deben reconocerse explícitamente. En primer lugar, las decisiones éticas documentadas corresponden al estado actual del sistema y no contemplan escenarios de escala masiva (millones de usuarios concurrentes) donde los dilemas podrían adquirir nuevas dimensiones. En segundo lugar, el marco de evaluación se ha centrado en la ética normativa occidental; futuras iteraciones podrían beneficiarse de perspectivas adicionales como la ética del cuidado o enfoques interculturales. En tercer lugar, no se ha realizado una validación empírica del impacto de estas decisiones en la experiencia del usuario final, lo que constituye una línea de trabajo futuro inmediato.

En última instancia, este informe demuestra que la ética no es un agregado opcional al ciclo de desarrollo de software, sino un componente constitutivo de la arquitectura misma. Como señalan Floridi y Cowls (2019), la ingeniería de sistemas de

información es, ante todo, una disciplina moral.

### Referencias

- [1] ACM. (2018). *ACM Code of Ethics and Professional Conduct*. Association for Computing Machinery. <https://www.acm.org/code-of-ethics>
- [2] Beauchamp, T. L., & Childress, J. F. (2019). *Principles of biomedical ethics* (8th ed.). Oxford University Press.
- [3] Floridi, L., & Cows, J. (2019). A unified framework of five principles for AI in society. *Harvard Data Science Review*, 1(1).  
<https://doi.org/10.1162/99608f92.8cd550d1>
- [4] Kant, I. (2012). *Fundamentación de la metafísica de las costumbres* (M. García Morente, Trad.). Porrúa. (Trabajo original publicado en 1785)
- [5] Maner, W. (1996). Unique ethical problems in information technology. *Science and Engineering Ethics*, 2(2), 137–154. <https://doi.org/10.1007/BF02583549>
- [6] Mill, J. S. (2001). *Utilitarianism* (2nd ed.). Hackett Publishing. (Trabajo original publicado en 1861)
- [7] Mittelstadt, B. D., Allo, P., Taddeo, M., Wachter, S., & Floridi, L. (2016). The ethics of algorithms: Mapping the debate. *Big Data & Society*, 3(2).  
<https://doi.org/10.1177/2053951716679679>
- [8] Nissenbaum, H. (2009). *Privacy in context: Technology, policy, and the integrity of social life*. Stanford University Press.
- [9] Rawls, J. (1971). *A theory of justice*. Harvard University Press.
- [10] Shneiderman, B. (2020). Bridging the gap between ethics and practice: Guidelines for reliable, safe, and trustworthy human-centered AI systems. *ACM Transactions on Interactive Intelligent Systems*, 10(4), 1–31.  
<https://doi.org/10.1145/3419764>



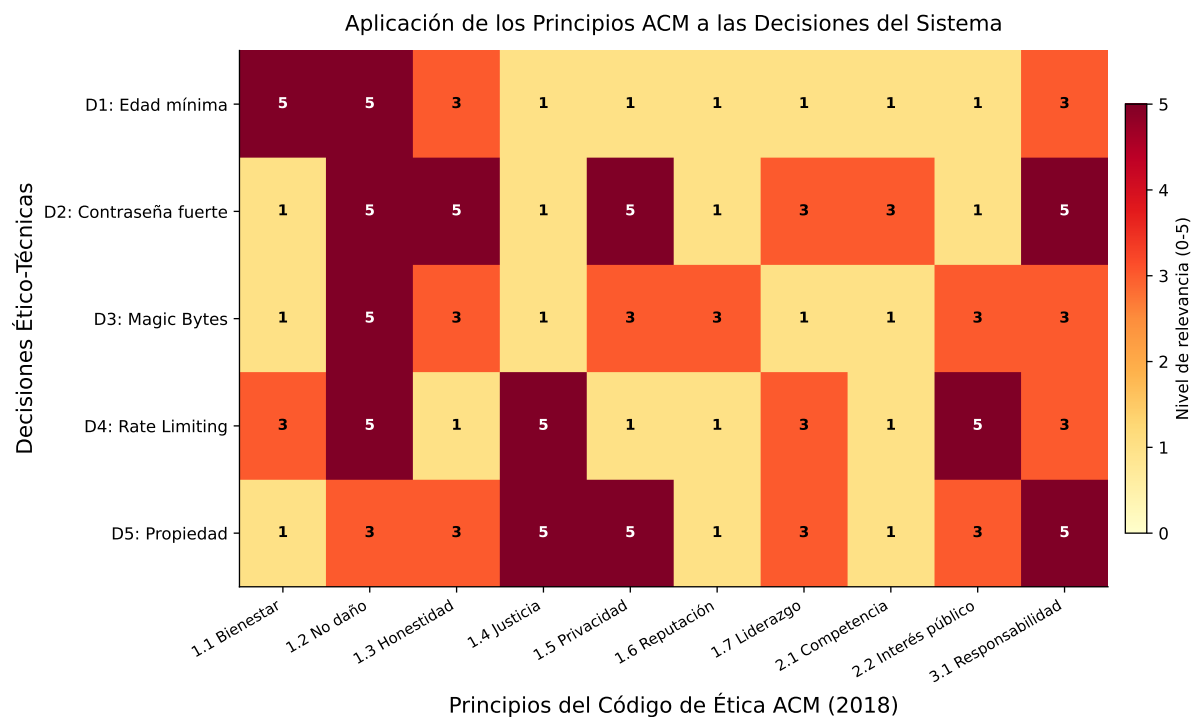
**Tabla 1***Matriz de Decisiones Ético-Técnicas del Sistema*

| # | Descripción                                                                            | Riesgo ético                                                    | Criterio                                                                       | Implementación                                                                                                             |
|---|----------------------------------------------------------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| 1 | Restricción de edad mínima (18 años) para registro                                     | Exposición de menores a contenido no regulado o inapropiado     | Deontología: imperativo de proteger al vulnerable                              | <code>RegisterPage.jsx</code> valida <code>max="YYYY-MM-DD"-18</code> años; backend almacena <code>fecha_nacimiento</code> |
| 2 | Validación de contraseña fuerte (8+ caracteres, mayúscula, minúscula, número, símbolo) | Robo de identidad y vulneración de privacidad                   | Responsabilidad profesional (ACM 1.3): garantizar seguridad de datos confiados | <code>Regex</code> en <code>front/back + hashing bcrypt</code> con salt en <code>security.py</code>                        |
| 3 | Filtros estrictos de contenido por verificación de <i>magic bytes</i>                  | Propagación de malware disfrazado de imágenes (esteganografía)  | No maleficencia (ACM 1.2): evitar que la plataforma sea vector de daño         | <code>_detect_mime()</code> en <code>pins.py</code> lee cabeceras hex antes de subir a S3                                  |
| 4 | <i>Rate limiting</i> (10 peticiones/minuto por IP)                                     | Ataques DoS o spam masivo que degradan la experiencia colectiva | Justicia distributiva (Rawls): acceso equitativo                               | Middleware <code>slowapi</code> en <code>main.py + limiter.py</code>                                                       |

**Tabla 2***Matriz de Implementación Verificable*

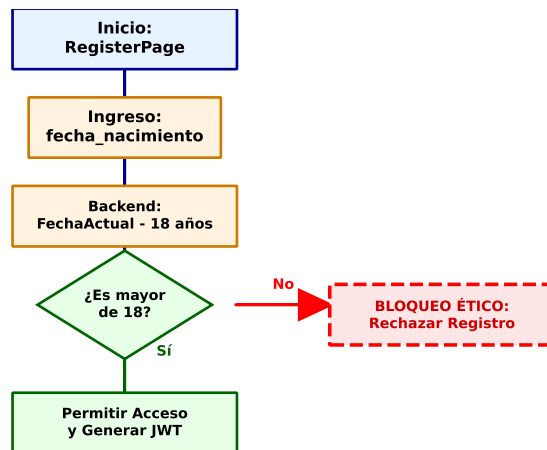
| Capa<br>Técnica              | Riesgo Ético a Mitigar                                    | Solución en Código                                                                                                                         |
|------------------------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Autenticación</b>         | Exposición de credenciales a scripts maliciosos (XSS)     | JWT emitido por <b>cookies</b><br><b>HttpOnly</b> ; el frontend nunca accede al token desde JavaScript ( <code>dependencies.py:41</code> ) |
| <b>Red y Tráfico</b>         | Ataques de spam que privan del servicio legítimo (DoS)    | Middleware <b>slowapi</b> : endpoints críticos a <b>10/minute</b> por IP ( <code>limiter.py; pins.py:155</code> )                          |
| <b>Identidad Cloud</b>       | Rastreo de datos personales por nombres de archivo en AWS | Sanitización con <b>UUIDv4</b> al transferir a S3; extensión limpia con regex ( <code>storage_service.py:38-49</code> )                    |
| <b>Validación Multimedia</b> | Subida de archivos maliciosos camuflados como imágenes    | Lectura de <b>magic bytes</b> en memoria; verificación de firma hex contra MIME permitidos ( <code>pins.py:20-29</code> )                  |
| <b>Auditoría</b>             | Falta de trazabilidad en acciones críticas                | Logger JSON estructurado con eventos de login, eliminación e intentos no autorizados ( <code>audit_log.py:1-39</code> )                    |

*Nota.* Cada fila contiene una referencia directa al archivo y línea de código donde la mitigación está implementada.

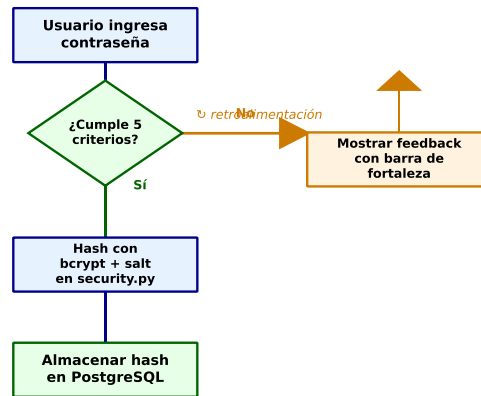


**Figura 1**

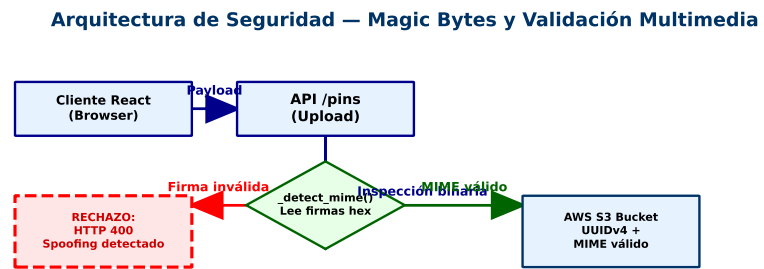
Mapa de calor de la relevancia de cada principio del Código ACM (2018) para las cinco decisiones ético-técnicas del sistema.

**Diagrama de Flujo Ético — Restricción de Edad****Figura 2**

*Diagrama de flujo para la restricción de edad. El bloqueo ético interrumpe el flujo comercial de registro.*

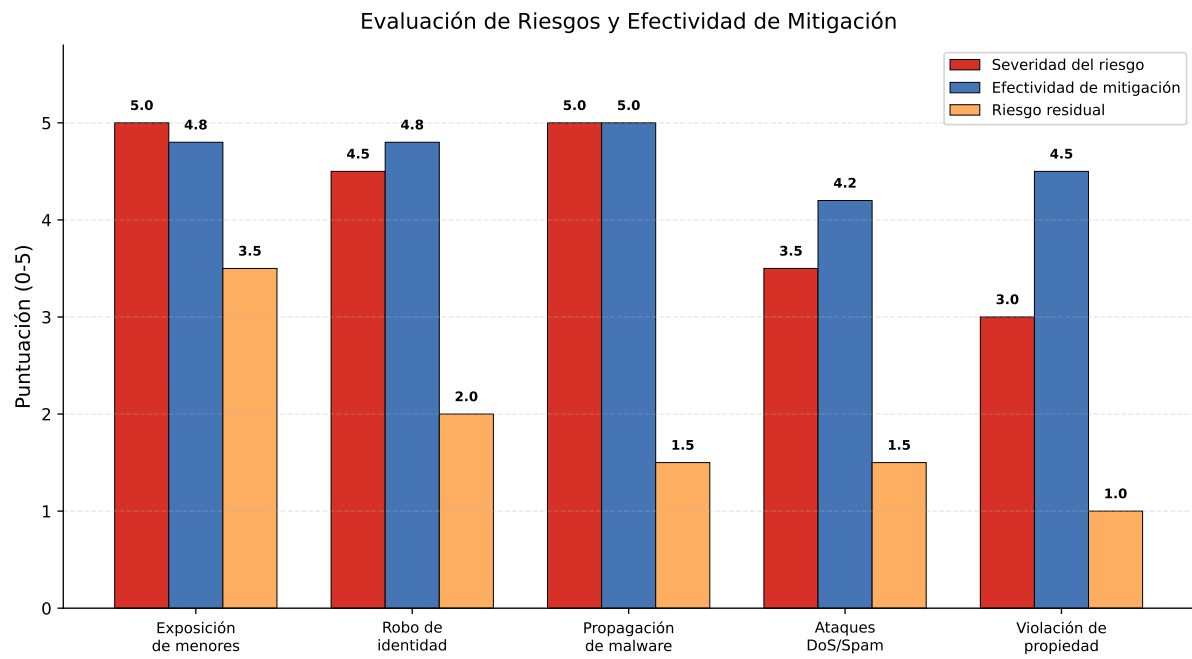
**Diagrama de Flujo — Validación y Hash de Contraseña****Figura 3**

*Ciclo de validación y hash de contraseña con retroalimentación en tiempo real.*



**Figura 4**

*Arquitectura de validación multimedia con detección de magic bytes en dos capas.*

**Figura 5**

*Comparación de severidad del riesgo ético, efectividad de mitigación y riesgo residual.*